

INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY (ICDFA)

FORMAL DIGITAL FORENSICS INVESTIGATION REPORT

SBT-DF203 - LAB 5: ARP Analysis and Detection Using TShark/Wireshark

Case Identifier	SBT-DF203-Lab5
Lead Examiner	Nebeuwa Ifeanyichukwu Raphael
Registration / ID No.	202520850LE
Course Code	SBT-DF203
Course Title	Basic Networking Skills for Digital Forensics
Lab Number	Lab 5
Lab Title	ARP Analysis and Detection Using TShark/Wireshark
Date of Submission	September 17, 2026
Primary Target System	Kali Linux Workstation (kali@kali) with isolated Windows 10 VM
Evidence Type	Network Packet Capture (.pcap/.pcapng) and associated analysis artifacts

Prepared for authorized academic forensic examination

1. Executive Summary

This report documents my performance of SBT-DF203 Lab 5, an ARP-focused network-forensics exercise. The examination was conducted in a controlled VMware environment using Kali Linux and a Windows 10 virtual machine. The practical workflow established an isolated host-only network, preserved a supplied ARP packet capture, verified its integrity, captured normal ARP resolution, extracted ARP fields with tshark, identified ARP request/reply pairs, summarized observed IP-to-MAC claims, and constructed an ARP timeline for detection analysis.

The live environment was configured on the 192.168.190.0/24 host-only network. Kali used 192.168.190.129/24 and Windows 10 used 192.168.190.128/24. A direct arping test from Kali received 3 of 3 replies from the Windows host, confirming working Layer-2/ARP communication. The earlier ICMP ping test received no replies; this was not treated as evidence of network failure because ARP resolution was subsequently confirmed.

Evidence integrity was verified by SHA-256. The original capture evidence/arp.pcap and its working copy working/arp_working.pcap both produced the identical digest 342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e, establishing that the analysis copy was bit-for-bit consistent with the preserved original.

The supplied ARP capture used for the detection analysis contained eight ARP frames. Frames 3 and 4 formed a request/reply exchange between 136.160.215.15 and 136.160.215.194, while frames 5 and 6 formed the reciprocal request/reply exchange. The observed IP-to-MAC claims were unique for the two analyzed IP addresses. No conflicting IP-to-MAC mapping or single-MAC claim for both analyzed hosts was observed in the supplied capture. Accordingly, the supplied capture does not independently establish an ARP-poisoning event.

Part D, the optional controlled simulation, was not performed because the assessment's instructor-provided arp.py script was not present in the available lab materials. No substitute poisoning script was introduced, preserving the provenance of the exercise and avoiding unsupported claims about a simulated attack.

2. Investigation Overview and Scope

2.1 Purpose and Objectives

- Establish a controlled ARP-analysis environment using Kali Linux and Windows 10.
- Preserve and hash the supplied ARP evidence before analysis.
- Observe and document normal ARP request/reply resolution.
- Extract ARP fields including opcode, protocol IPs, and hardware MAC addresses.
- Identify IP-to-MAC claims and examine them for conflicts.
- Construct a packet-level ARP timeline and apply detection logic.
- Document limitations and clearly distinguish observed evidence from unperformed simulation activity.

2.2 Evidence Media and Environment

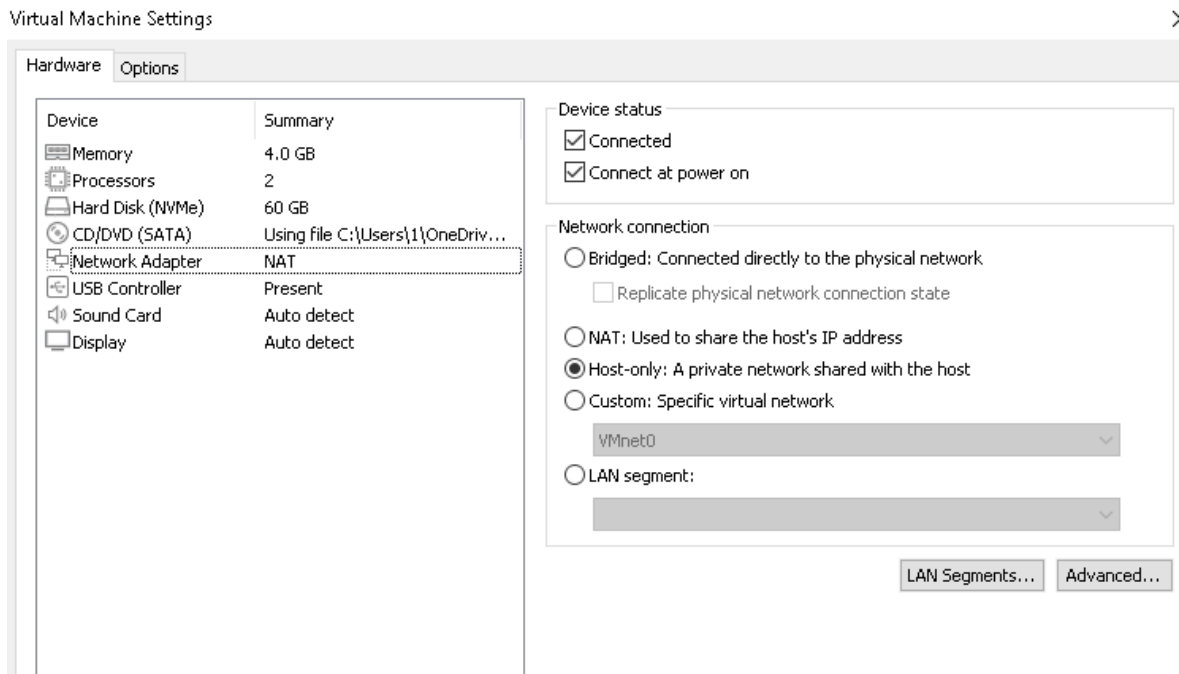


Figure 1: VMware virtual-machine network configuration showing the host-only network used for the lab.

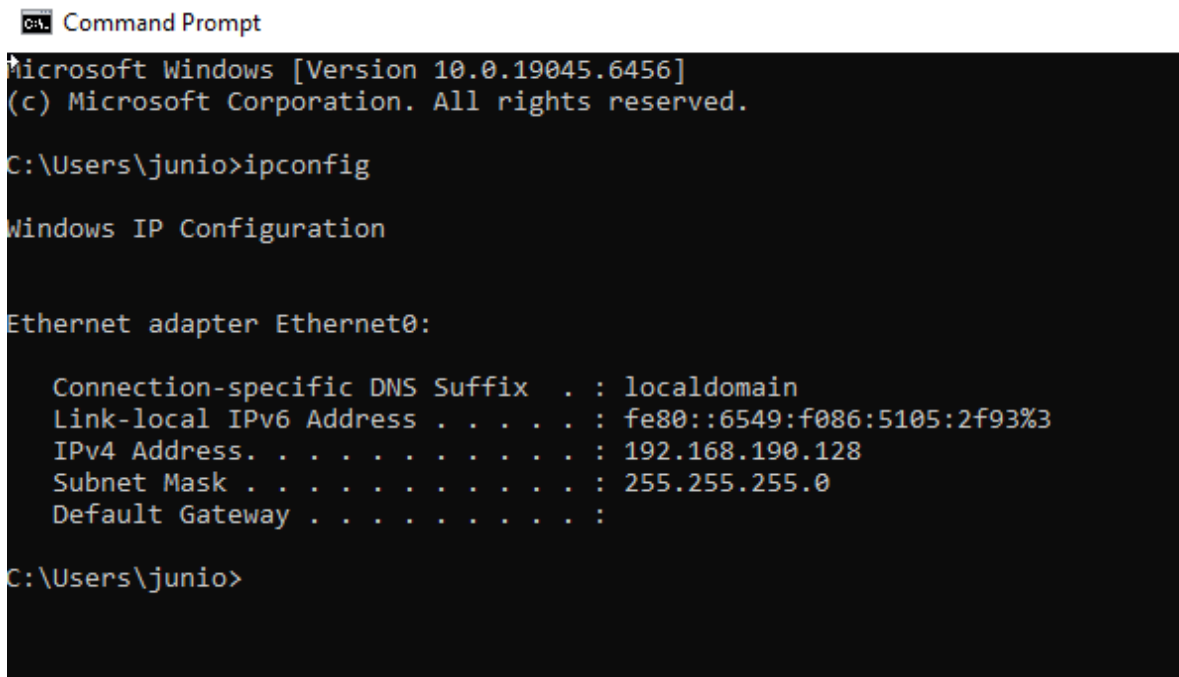


Figure 2: Windows 10 network configuration and ARP table for the lab host.

```

Session  Actions  Edit  View  Help

(kali㉿kali)-[~]
$ ip -br address
lo                UNKNOWN      127.0.0.1/8 ::1/128
eth0              UP          192.168.190.129/24 fe80::bd78:42d5:f457:6742/64

(kali㉿kali)-[~]
$ ip route
192.168.190.0/24 dev eth0 proto kernel scope link src 192.168.190.129 metric 100

(kali㉿kali)-[~]
$ ipconfig
Command 'ipconfig' not found, did you mean:
  command 'iwconfig' from deb wireless-tools
  command 'iconfig' from deb ipmiutil
  command 'ifconfig' from deb net-tools
  command 'hipconfig' from deb hipcc
Try: sudo apt install <deb name>

(kali㉿kali)-[~]
$ ip neigh show
192.168.190.1 dev eth0 lladdr 00:50:56:c0:00:01 REACHABLE
192.168.190.254 dev eth0 lladdr 00:50:56:e4:6d:6b STALE
fe80::e7c9:a0ad:7bc6:d1b7 dev eth0 lladdr 00:50:56:c0:00:01 STALE

(kali㉿kali)-[~]
$ ping -c 4 192.168.190.128
PING 192.168.190.128 (192.168.190.128) 56(84) bytes of data.

— 192.168.190.128 ping statistics —
4 packets transmitted, 0 received, 100% packet loss, time 3052ms

(kali㉿kali)-[~]

```

Figure 3: Kali Linux interface and route information used to confirm the lab network configuration.

Component	Observed configuration
Kali Linux	192.168.190.129/24, eth0
Windows 10	192.168.190.128/24, Ethernet0
Virtual network	VMware Host-only network, 192.168.190.0/24
Primary evidence	evidence/arp.pcap
Working copy	working/arp_working.pcap
Tools	tshark, Wireshark, arping, ip, ping, sha256sum

2.3 Evidence Handling and Working Directory

The lab workspace was organized into evidence, working, exported, reports, screenshots and scripts directories. The original supplied capture was retained under evidence/, while analysis was performed against the preserved working copy. The scripts directory was checked and found empty; a recursive search confirmed that no arp.py file was available under the home directory or Lab 5 workspace.

3. Evidence Preparation and Baseline Integrity

```
Session  Actions  Edit  View  Help
(kali@kali)-[~/SBT-DF203-Lab5]
$ sudo apt update
Get:2 https://repo.protonvpn.com/debian stable InRelease [2,967 B]
Get:3 https://repo.protonvpn.com/debian stable/main amd64 Packages [22.1 kB]
Get:4 https://repo.protonvpn.com/debian stable/main all Packages [234 kB]
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:5 http://kali.download/kali kali-rolling/main amd64 Packages [21.6 MB]
Get:6 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.9 MB]
Get:7 http://kali.download/kali kali-rolling/contrib amd64 Packages [116 kB]
Get:8 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [270 kB]
Get:9 http://kali.download/kali kali-rolling/non-free amd64 Packages [183 kB]
Get:10 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [915 kB]
Get:11 http://kali.download/kali kali-rolling/non-free-firmware amd64 Packages [16.0 kB]
Get:12 http://kali.download/kali kali-rolling/non-free-firmware amd64 Contents (deb) [41.8 kB]
Fetched 77.3 MB in 5min 31s (234 kB/s)
1914 packages can be upgraded. Run 'apt list --upgradable' to see them.

(kali@kali)-[~/SBT-DF203-Lab5]
$ sudo apt install -y wireshark tshark python3-scapy net-tools
wireshark is already the newest version (4.6.6-1).
tshark is already the newest version (4.6.6-1).
python3-scapy is already the newest version (2.7.0+dfsg1-1).
net-tools is already the newest version (2.10-2).
net-tools set to manually installed.
The following packages were automatically installed and are no longer required:
 libimath-3-1-29t64  libpython3.13-dev  python3.13  rpcsvc-proto
 libmbcrypto16      libraw23t64  python3.13-dev
 libpostal-data     libvidstab1.1  python3.13-minimal
 libpostal1         libzxing3     python3.13-venv
Use 'sudo apt autoremove' to remove them.
```

Figure 4: Kali Linux update and package-installation output showing Wireshark, TShark and supporting tools prepared for the lab.

The supplied ARP capture was preserved before analysis. A working copy was created so that analysis operations could be performed without modifying the original evidence object.

Evidence item	SHA-256
evidence/arp.pcap	342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e
working/arp_working.pcap	342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e

The identical hashes provide the primary integrity check for the working copy. This supports the conclusion that the analysis copy is a bit-for-bit duplicate of the original capture.

```
Session Actions Edit View Help

(kali㉿kali)-[~/SBT-DF203-Lab5]
$ find evidence working reports -maxdepth 1 -type f -printf '%p\n' | sort
evidence/arp.pcap
evidence/normal_arp.pcapng
evidence/partD_arp_simulation.pcapng
reports/arp_capture_hashes.txt
reports/arp_clean_kali.txt
reports/arp_replies.tsv
reports/arp_table_after_ping.txt
reports/arp_table_initial.txt
reports/full_arp_timeline.tsv
reports/interfaces.txt
reports/ip_mac_claims.txt
reports/normal_arp_fields.tsv
reports/partE_arp_analysis.txt
reports/partE_ip_mac_pairs.txt
reports/routes.txt
reports/unicast_arp_replies.tsv
working/arp_working.pcap

(kali㉿kali)-[~/SBT-DF203-Lab5]
$ cat reports/arp_capture_hashes.txt
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e  evidence/arp.pcap
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e  working/arp_working.pcap

(kali㉿kali)-[~/SBT-DF203-Lab5]
$
```

Figure 5: Terminal evidence showing the preserved evidence/working files and matching SHA-256 values.

3.1 Baseline ARP State in the Isolated Lab

Before the later analysis steps, the two virtual machines were placed on the same VMware host-only network. Windows reported 192.168.190.128/24 and Kali reported 192.168.190.129/24. Windows' ARP table contained the dynamic mapping 192.168.190.129 → 00-0c-29-6c-fb-53. Kali subsequently learned the Windows mapping 192.168.190.128 → 00:0c:29:77:f3:67.

3.2 ARP Reachability Verification

An ICMP ping from Kali to 192.168.190.128 produced 100% packet loss. This did not prevent the ARP examination. A subsequent arping test to 192.168.190.128 returned three replies from 00:0c:29:77:f3:67, with 0% unanswered. The result confirms that the two VMs could resolve each other's Layer-2 address even though ICMP echo replies were not observed.

4. Part A - Normal ARP Resolution

```
Session Actions Edit View Help
(kali@kali)-[~/SBT-DF203-Lab5]
$ tshark -r evidence/normal_arp.pcapng -Y 'arp' -T fields \
> -e frame.number -e frame.time -e eth.src -e eth.dst -e arp.opcode \
> -e arp.src.proto_ipv4 -e arp.src.hw_mac -e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
> | tee reports/normal_arp_fields.tsv
1      2026-09-17T12:54:57.698655983-0400      00:50:56:c0:00:08      ff:ff:ff:ff:ff:ff:
ff:ff  1      192.168.146.1      00:50:56:c0:00:08      192.168.146.2      00:00:00:00:00:00:
00:00
2      2026-09-17T12:54:58.761886497-0400      00:0c:29:6c:fb:53      ff:ff:ff:ff:ff:ff:
ff:ff  1      192.168.146.132 00:0c:29:6c:fb:53      192.168.146.2      00:00:00:00:00:00:
00:00
3      2026-09-17T12:54:58.762045334-0400      00:50:56:e9:9c:8e      00:0c:29:6c:fb:53
fb:53  2      192.168.146.2      00:50:56:e9:9c:8e      192.168.146.132 00:0c:29:6c:fb:53
fb:53
4      2026-09-17T12:54:59.209751291-0400      00:50:56:c0:00:08      ff:ff:ff:ff:ff:ff:
ff:ff  1      192.168.146.1      00:50:56:c0:00:08      192.168.146.2      00:00:00:00:00:00:
00:00
5      2026-09-17T12:55:00.209475055-0400      00:50:56:c0:00:08      ff:ff:ff:ff:ff:ff:
ff:ff  1      192.168.146.1      00:50:56:c0:00:08      192.168.146.2      00:00:00:00:00:00:
00:00
6      2026-09-17T12:55:01.210462285-0400      00:50:56:c0:00:08      ff:ff:ff:ff:ff:ff:
ff:ff  1      192.168.146.1      00:50:56:c0:00:08      192.168.146.2      00:00:00:00:00:00:
00:00
7      2026-09-17T12:55:05.246436984-0400      00:50:56:c0:00:08      ff:ff:ff:ff:ff:ff:
ff:ff  1      192.168.146.1      00:50:56:c0:00:08      192.168.146.2      00:00:00:00:00:00:
00:00
8      2026-09-17T12:55:06.201449750-0400      00:50:56:c0:00:08      ff:ff:ff:ff:ff:ff:
ff:ff  1      192.168.146.1      00:50:56:c0:00:08      192.168.146.2      00:00:00:00:00:00:
00:00
9      2026-09-17T12:55:07.201335108-0400      00:50:56:c0:00:08      ff:ff:ff:ff:ff:ff:
ff:ff  1      192.168.146.1      00:50:56:c0:00:08      192.168.146.2      00:00:00:00:00:00:
00:00
```

Figure 6: TShark field extraction from the normal ARP capture used to establish the baseline.

The normal ARP capture was examined to document the expected request/reply process before considering any anomalous behaviour. The capture was filtered with tshark using the ARP protocol and relevant fields were exported to reports/normal_arp_fields.tsv.

Frame	Opcode	Sender IP	Sender MAC	Target IP	Target MAC
1	1	192.168.146.1	00:50:56:c0:00:08	192.168.146.2	00:00:00:00:00:00
2	1	192.168.146.132	00:0c:29:6c:fb:53	192.168.146.2	00:00:00:00:00:00
3	2	192.168.146.2	00:50:56:e9:9c:8e	192.168.146.132	00:0c:29:6c:fb:53
4	1	192.168.146.1	00:50:56:c0:00:08	192.168.146.2	00:00:00:00:00:00

The normal capture shows a conventional ARP request (opcode 1) followed by an ARP reply (opcode 2), with the reply supplying the sender's hardware address for the requested IP. Repeated broadcast requests from the gateway-like address are also present. These observations form the behavioural baseline used later in detection analysis.

4.1 Baseline Interpretation

- ARP requests use opcode 1 and normally have an unknown target hardware address represented as 00:00:00:00:00:00.
- ARP replies use opcode 2 and provide a concrete sender hardware address.
- A request/reply pair provides the expected evidence for ordinary address resolution.
- A later detection review should distinguish such solicited replies from replies that appear without a recent corresponding request.

5. Part B - ARP Field Extraction and Reply Analysis

```
(kali㉿kali)-[~/SBT-DF203-Lab5]
$ PCAP=working/arp_working.pcap

(kali㉿kali)-[~/SBT-DF203-Lab5]
$ tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields \
> -e frame.number -e frame.time_epoch -e eth.src -e eth.dst \
> -e arp.src.proto_ipv4 -e arp.src.hw_mac -e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
> | tee reports/arp_replies.tsv
4      1678591364.904123642    00:50:56:86:02:65    00:50:56:86:cb:fc    136.
160.215.194    00:50:56:86:02:65    136.160.215.15    00:50:56:86:cb:fc
6      1678591369.943043799    00:50:56:86:cb:fc    00:50:56:86:02:65    136.
160.215.15    00:50:56:86:cb:fc    136.160.215.194    00:50:56:86:02:65

(kali㉿kali)-[~/SBT-DF203-Lab5]
$ | tee reports/arp_replies.tsv
zsh: parse error near `|'

(kali㉿kali)-[~/SBT-DF203-Lab5]
$ cat reports/arp_replies.tsv
4      1678591364.904123642    00:50:56:86:02:65    00:50:56:86:cb:fc    136.
160.215.194    00:50:56:86:02:65    136.160.215.15    00:50:56:86:cb:fc
6      1678591369.943043799    00:50:56:86:cb:fc    00:50:56:86:02:65    136.
160.215.15    00:50:56:86:cb:fc    136.160.215.194    00:50:56:86:02:65

(kali㉿kali)-[~/SBT-DF203-Lab5]
$
```

Figure 7: TShark extraction of ARP reply frames from the working capture.

The working capture was processed with tshark to extract frame number, timestamp, Ethernet source/destination, ARP opcode, sender protocol address, sender hardware address, target protocol address and target hardware address. The resulting data were retained in reports/arp_replies.tsv and related TSV reports.

Frame	Opcode	Source IP	Source MAC	Destination IP	Destination MAC
4	2	136.160.215.194	00:50:56:86:02:65	136.160.215.15	00:50:56:86:cb:fc
6	2	136.160.215.15	00:50:56:86:cb:fc	136.160.215.194	00:50:56:86:02:65

Frames 4 and 6 are the two ARP replies in the eight-frame supplied capture. Each reply corresponds to a preceding request: frame 4 follows frame 3, and frame 6 follows frame 5. This pairing is consistent with ordinary ARP resolution rather than an unsolicited-reply pattern.

5.1 IP-to-MAC Claim Summary

```
(kali㉿kali)-[~/SBT-DF203-Lab5]
$ tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields \
> -e arp.src.proto_ipv4 -e arp.src.hw_mac \
> | sort | uniq -c | sort -nr \
pipe pipe pipe> | tee reports/ip_mac_claims.txt
    1 136.160.215.194 00:50:56:86:02:65
    1 136.160.215.15  00:50:56:86:cb:fc

(kali㉿kali)-[~/SBT-DF203-Lab5]
$ cat reports/ip_mac_claims.txt
    1 136.160.215.194 00:50:56:86:02:65
    1 136.160.215.15  00:50:56:86:cb:fc

(kali㉿kali)-[~/SBT-DF203-Lab5]
$
```

Figure 8: IP-to-MAC claim summary extracted from the observed ARP replies.

IP address	Observed MAC	Observed claims
136.160.215.15	00:50:56:86:cb:fc	1
136.160.215.194	00:50:56:86:02:65	1

The claim inventory contains one observed MAC address for each analyzed IP address. No duplicate/conflicting mapping is present in this capture for these two addresses.

6. Part C - Full ARP Timeline

```
(kali㉿kali)-[~/SBT-DF203-Lab5]
$ tshark -r "$PCAP" -Y 'arp' -T fields \
> -e frame.number -e arp.opcode \
> -e arp.src.proto_ipv4 -e arp.src.hw_mac \
> -e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
> | column -t
 1 1 136.160.215.1 00:1b:17:00:0a:30 136.160.215.1 ff:ff:ff:ff:ff:ff
 2 1 136.160.215.1 00:1b:17:00:0a:30 136.160.215.199 ff:ff:ff:ff:ff:ff
 3 1 136.160.215.15 00:50:56:86:cb:fc 136.160.215.194 00:00:00:00:00:00
 4 2 136.160.215.194 00:50:56:86:02:65 136.160.215.15 00:50:56:86:cb:fc
 5 1 136.160.215.194 00:50:56:86:02:65 136.160.215.15 00:00:00:00:00:00
 6 2 136.160.215.15 00:50:56:86:cb:fc 136.160.215.194 00:50:56:86:02:65
 7 1 136.160.215.1 00:1b:17:00:0a:30 136.160.215.1 ff:ff:ff:ff:ff:ff
 8 1 136.160.215.1 00:1b:17:00:0a:30 136.160.215.183 ff:ff:ff:ff:ff:ff

(kali㉿kali)-[~/SBT-DF203-Lab5]
$
```

Figure 9: Full eight-frame ARP timeline showing requests, replies and the associated IP/MAC fields.

The full timeline extracted from the supplied working capture contains eight ARP frames. The following table condenses the relevant fields for forensic interpretation.

Frame	Time (epoch)	Op	Sender IP	Sender MAC	Target IP
1	1678591323.059513401	1	136.160.215.1	00:1b:17:00:0a:30	136.160.215.1
2	1678591358.192015486	1	136.160.215.1	00:1b:17:00:0a:30	136.160.215.199
3	1678591364.903581555	1	136.160.215.15	00:50:56:86:cb:fc	136.160.215.194
4	1678591364.904123642	2	136.160.215.194	00:50:56:86:02:65	136.160.215.15
5	1678591369.943023192	1	136.160.215.194	00:50:56:86:02:65	136.160.215.15
6	1678591369.943043799	2	136.160.215.15	00:50:56:86:cb:fc	136.160.215.194
7	1678591383.059262685	1	136.160.215.1	00:1b:17:00:0a:30	136.160.215.1
8	1678591386.191965551	1	136.160.215.1	00:1b:17:00:0a:30	136.160.215.183

The most diagnostically useful sequence is frames 3–6. The first request from 136.160.215.15 to 136.160.215.194 is followed almost immediately by the corresponding reply, and the reciprocal request/reply pair follows several seconds later. There is no observed reply from an unrelated MAC inserted between these legitimate pairs.

6.1 Detection Logic

Indicator	Evidence observed	Interpretation
IP-to-MAC change	No conflicting mapping in the analyzed claim set	Not demonstrated
Repeated unsolicited replies	Reply frames 4 and 6 each follow a request	Not demonstrated
One MAC claiming both hosts	15 → cb:fc; 194 → 02:65	Not demonstrated
Other anomalies correlated with ARP	No additional anomalous event established from supplied ARP data	Not established

7. Part D - Controlled ARP Simulation (Not Performed)

```

(kali㉿kali)-[~/SBT-DF203-Lab5]
$ tshark -r "$PCAP" -Y 'arp' -T fields \
> -e arp.src.proto_ipv4 -e arp.src.hw_mac \
> | sort -u
136.160.215.1    00:1b:17:00:0a:30
136.160.215.15  00:50:56:86:cb:fc
136.160.215.194 00:50:56:86:02:65

(kali㉿kali)-[~/SBT-DF203-Lab5]
$

```

Figure 10: IP-to-MAC pair extraction used for the Part E analysis.

The controlled simulation portion was not performed. The assessment procedure called for an instructor-provided arp.py script. The lab workspace's scripts directory was empty, and a recursive search under the user's home directory returned no arp.py file. Because the required course-supplied script was unavailable, no substitute script was introduced.

This is a procedural limitation rather than an observed network finding. The report therefore makes no claim that ARP poisoning occurred in the live Windows/Kali environment. The existing Part D capture filename is retained as an artifact of the attempted setup, but it is not treated as evidence of a successful poisoning simulation.

```
(kali㉿kali)-[~/SBT-DF203-Lab5]
└─$ tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields \
> -e arp.src.proto_ipv4 -e arp.src.hw_mac \
> | sort | uniq -c | sort -nr \
pipe pipe pipe> | tee reports/ip_mac_claims.txt
    1 136.160.215.194 00:50:56:86:02:65
    1 136.160.215.15  00:50:56:86:cb:fc

(kali㉿kali)-[~/SBT-DF203-Lab5]
└─$ cat reports/ip_mac_claims.txt
    1 136.160.215.194 00:50:56:86:02:65
    1 136.160.215.15  00:50:56:86:cb:fc

(kali㉿kali)-[~/SBT-DF203-Lab5]
└─$
```

Figure 11: Consolidated IP-to-MAC claims supporting the Part E detection analysis.

This approach preserves evidentiary accuracy: a forensic report should distinguish an observed event from a procedure that could not be executed. The remaining analysis is based on the supplied capture and the independently verified normal-resolution observations.

8. Evidence Inventory and Reproducibility

Path	Role / contents
evidence/arp.pcap	Supplied ARP packet capture; preserved original
working/arp_working.pcap	Working analysis copy; SHA-256 matches original
evidence/normal_arp.pcapng	Normal ARP-resolution capture
reports/normal_arp_fields.tsv	Extracted normal ARP fields
reports/arp_replies.tsv	Extracted ARP replies
reports/ip_mac_claims.txt	IP-to-MAC claim summary
reports/unicast_arp_replies.tsv	Unicast ARP reply analysis
reports/full_arp_timeline.tsv	Full ARP timeline
reports/arp_table_initial.txt	Initial Kali ARP/neighbour state
reports/arp_table_after_ping.txt	Post-ping Kali ARP/neighbour state
reports/arp_clean_kali.txt	Clean Kali neighbour-state record
reports/partE_arp_analysis.txt	Part E ARP analysis output
reports/partE_ip_mac_pairs.txt	Unique IP-to-MAC pairs
reports/arp_capture_hashes.txt	SHA-256 integrity record

9. Required Forensic Findings

Finding	Observation
Network configuration	Kali 192.168.190.129/24 and Windows 10 192.168.190.128/24 on the same 192.168.190.0/24 host-only network.
ARP reachability	Kali arping to 192.168.190.128 returned 3/3 replies from 00:0c:29:77:f3:67.
Evidence integrity	evidence/arp.pcap and working/arp_working.pcap both hash to 342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e.
Normal ARP behaviour	Normal capture contains request/reply traffic consistent with address resolution.
Analyzed ARP replies	Frames 4 and 6 are replies corresponding to frames 3 and 5 respectively.
IP-to-MAC claims	136.160.215.15 maps to 00:50:56:86:cb:fc; 136.160.215.194 maps to 00:50:56:86:02:65.
Conflict indicator	No conflicting IP-to-MAC mapping was observed in the analyzed claim set.
Unsolicited-reply indicator	No demonstrated unsolicited ARP reply; both analyzed replies follow requests.
Single-MAC dual-claim indicator	No single observed MAC claims both analyzed IP addresses.
Part D status	Controlled simulation not performed because the required instructor-provided arp.py was unavailable.

10. Conclusion and Findings

The examination established a controlled ARP-analysis workflow, preserved the supplied capture, verified working-copy integrity, documented normal ARP resolution, and extracted the packet-level fields required for forensic interpretation. The live Kali/Windows environment was successfully placed on an isolated host-only network, and ARP reachability was verified independently of ICMP.

The supplied capture shows conventional ARP request/reply behaviour. The two analyzed host addresses have distinct, stable observed MAC claims in the capture, and the reply frames examined are directly associated with preceding requests. On the evidence available for this exercise, the capture does not independently establish ARP poisoning.

The optional controlled simulation was not performed because the required instructor-provided arp.py script was unavailable. This limitation is explicitly recorded so that the report does not imply an event or procedure that was not actually observed. The preserved evidence, extracted reports and integrity record provide a reproducible basis for the completed portions of the lab.

11. Limitations

- The instructor-provided arp.py required for the optional controlled simulation was unavailable in the supplied workspace.
- ICMP ping from Kali to Windows received no echo replies; ARP reachability was instead verified using arping.
- The report relies on the packet evidence and terminal outputs actually generated during the lab; no unsupported attack-state conclusion has been added.

12. Appendix A - Principal Commands Used

```
ip -br address
```

```
ip route
```

```
ip neigh show
```

```
ping -c 1 192.168.190.128
```

```
sudo arping -I eth0 -c 3 192.168.190.128
```

```
tshark -r working/arp_working.pcap -Y 'arp' -T fields ...
```

```
tshark -r working/arp_working.pcap -Y 'arp.opcode==2' -T fields ...
```

```
sha256sum evidence/arp.pcap working/arp_working.pcap | tee reports/arp_capture_hashes.txt
```

```
find ~ -type f -name 'arp.py' 2>/dev/null
```

13. Appendix B - Evidence Interpretation Note

The distinction between normal ARP behaviour and poisoning indicators is central to this examination. A normal request/reply pair demonstrates address resolution. A poisoning conclusion would require additional evidence such as a conflicting mapping, unsolicited or repeated replies, or a MAC address claiming multiple relevant IP addresses. None of those indicators was demonstrated by the supplied ARP capture analysed here.